

Loop unfolding (also called loop unrolling) is a compiler optimization technique where the loop body is replicated multiple times to reduce the overhead of loop control instructions like incrementing counters and checking conditions. For example, an unoptimized loop might be:

```
for (i = 0; i < 4; i++) {  
    A[i] = A[i] * 2;  
}
```

After loop unfolding by a factor of 2, the optimized code becomes:

```
for (i = 0; i < 4; i += 2) {  
    A[i] = A[i] * 2;  
    A[i + 1] = A[i + 1] * 2;  
}
```

The unoptimized intermediate code would repeatedly execute load, multiply, and store operations for each iteration, while the optimized version performs twice as many operations per loop, cutting the loop control cost in half. The main benefit is better CPU pipeline performance and reduced branching overhead, improving execution speed and resource utilization, especially for compute-intensive loops.